

Mamba’s key move: make a fast memory choose

Mamba’s contribution is a **selective state-space model**: a recurrent sequence layer whose compact running memory can decide, from each input token, what to keep and what to discard. That matters because the paper frames long-sequence modeling as a compression problem: attention keeps the whole past context explicitly, while a recurrent model is efficient precisely because it must summarize the past in a finite state. Mamba aims to give that summary content-dependent control without giving up linear scaling with sequence length.

The efficient starting point: a state-space model

A structured state-space model (SSM), specifically the paper’s S4 family, maintains a latent state: an internal vector that is updated one sequence element at a time and then read out as an output. In the discrete version, the new state h_t is formed from the previous state h_{t-1} plus the current input x_t ; the output y_t is read from that state.

The useful trick in prior SSMs is that their dynamics do not change over time. The parameters Δ , A , B , and C are fixed at every position, a property called **linear time invariance** (LTI). Here, A controls how the old state evolves, B controls how input enters the state, C controls how state becomes output, and Δ participates in converting the continuous-time formulation to the discrete update used on tokens.

With fixed dynamics, the same transformation has two equivalent computation views: step-by-step recurrence or one global convolution over the sequence. Earlier SSMs commonly used convolution during training, when the full sequence is available and parallel work is valuable, then used recurrence for autoregressive inference. That pairing is the source of their appeal: long-range sequence processing with linear or near-linear scaling.

Why fixed memory rules miss selective copying

That efficiency has a blind spot. A fixed update rule can’t decide that *this particular token* is worth retaining while the next one is noise. The paper illustrates the gap with **Selective Copying**: relevant tokens occur at variable positions among irrelevant ones, so the model must recognize content and filter noise rather than merely count a fixed delay.

A static convolution can handle the ordinary copying task when the spacing is constant, but variable spacing prevents a fixed kernel from lining up inputs and outputs. From the recurrence view, fixed transition parameters likewise cannot change what enters the state based on the input. The paper’s broader claim is not that finite-state models are inherently ineffective; it is that an efficient finite state needs selectivity if it is to compress a context while preserving the information needed later.

The method: an SSM whose controls depend on the token

Mamba changes S4 into a selective SSM, abbreviated **S6**, by making Δ , B , and C functions of the input at each sequence position. Those quantities therefore gain a sequence-length dimension: the SSM becomes time-varying rather than LTI.

The practical intuition is straightforward. Δ determines how much the update favors the current input versus the existing state: a small Δ preserves the state and largely ignores the input, while a large Δ resets toward the current input. Meanwhile, making B selective gives finer control over admitting the current input into memory, and making C selective controls how the state is exposed in the output.

There is a precise connection to familiar recurrent gates, but it is a special case—not the entire model. If the state has dimension $N = 1$, $A = -1$, $B = 1$, and the paper’s stated input projection and softplus conditions hold, the update becomes

$$h_t = (1 - g_t)h_{t-1} + g_tx_t,$$

where g_t is a sigmoid of a learned linear function of x_t . In that case, the gate interpolates between the old state and new input. The paper uses this connection to motivate its parameterization of Δ .

This differs from simply adding an elementwise multiplicative gate elsewhere in a network. The authors argue that such architectural gating need not affect propagation along the sequence axis, whereas selection changes the recurrent dynamics that determine what survives through time.

Recovering speed after losing convolution

Selection breaks the convenient convolution shortcut: once Δ , B , and C vary by token, the model is no longer equivalent to the global convolution used by LTI SSMs. The engineering half of the paper is therefore essential, not incidental.

Its **selective scan** combines kernel fusion, a parallel scan, and recomputation. Instead of writing the large expanded per-token state quantities to GPU high-bandwidth memory (HBM), the implementation loads the needed SSM parameters into faster SRAM—fast on-chip scratch memory—then discretizes and runs the recurrence there, materializing the expanded state locally rather than in HBM before writing only the final outputs back to HBM. A work-efficient parallel scan addresses the recurrence’s apparent sequentiality. During training, intermediate states are recomputed during backpropagation instead of stored, reducing the storage burden.

Mamba then places this selective SSM inside one repeated block. The architecture combines elements previously separated between an H3-style SSM block and a Transformer-style multilayer perceptron (MLP) block; it stacks the resulting homogeneous Mamba block with normalization and residual connections. This is an attention-free architecture, but the paper’s central novelty is the selective SSM plus the implementation that makes it practical—not just the block rearrangement.

What the reported evidence supports

The language scaling experiment trained models of roughly 125M to 1.3B parameters on the Pile under the stated Chinchilla protocol. In that range, the paper reports that Mamba matched its strong Transformer++ recipe, especially as sequence length grew, and outperformed the other attention-free models in the reported comparisons. The full 8K-context results omit RWKV and RetNet because the available implementations ran out of memory or required unrealistic computation, so that longer-context comparison does not cover those two baselines throughout. Those are controlled results in the reported size range, not evidence that it wins at every language-model scale.

The ablations help separate selection from architecture. At about 350M parameters in the paper’s language-model setting, swapping LTI inner layers for S6 produced a large perplexity improvement, while the simpler Mamba block performed similarly to H3. The selective-parameter ablation reports Δ as the single most important control, yet the all-three-selective configuration was best among the listed settings:

Selective parameters	Reported perplexity
none	10.93
B only	10.15
C only	9.98
Δ only	9.81
Δ , B , and C	8.71

The speed results also need their labels attached. For the core scan benchmark on an A100 80GB PCIe GPU, with state expansion $N = 16$, batch size 1, BF16 inputs, and the described range of sequence lengths, the fused scan surpassed FlashAttention-2 beyond length 2K. That core-operation comparison omits surrounding costs such as attention’s QKV projections and global-convolution kernel computation. In a separate end-to-end inference benchmark—2048-token prompts, 128 generated tokens, varying batch size, on the same A100—the paper reports 4–5 $\times$ higher throughput than similarly sized Transformers, attributing it to avoiding a key-value cache and hence supporting larger batches.

A useful mechanism, not a universal replacement

The paper establishes a specific trade: replace static, efficient filtering with input-dependent filtering, then recover practical execution through a hardware-aware scan. Its results support that trade for the reported language scaling range and show that selectivity, particularly Δ together with B and C , matters in the tested ablations.

But selection isn't automatically better for every sequence. On long-form audio waveform ablations, changing S4 to S6 significantly hurt performance; the authors connect this to audio's smooth, uniformly sampled character and LTI models' matching inductive bias. They also explicitly leave larger-scale performance and the associated engineering requirements unassessed, since their evaluation is below the size of many strong open language models. So the evidence supports Mamba as a compelling selective recurrent backbone under its tested conditions—not a blanket verdict against attention or time-invariant SSMs.